Vector + Map Syntax — Memorize Cold

This is the actual blocker, not logic. Type every line below by hand 3 times tonight.

VECTOR

Declare + Initialize

```
vector<int> v;                    // empty
vector<int> v = {1,2,3};          // with values
vector<int> v(5);                 // size 5, all 0
vector<int> v(5, 10);             // size 5, all filled with 10
vector<vector<int>> grid(3, vector<int>(4, 0));  // 3x4 grid of 0s
```

Add / Remove

```
v.push_back(10);       // add to end
v.pop_back();           // remove last
v.insert(v.begin(), 5); // insert 5 at front
v.erase(v.begin());     // remove first element
```

Access

```
v[0]              // first element
v.front()          // same as v[0]
v.back()           // last element
v.size()           // number of elements
v.empty()          // true if size == 0
```

Loop through vector — 3 ways

```
for (int i = 0; i < v.size(); i++) cout << v[i];     // index-based

for (int x : v) cout << x;                            // range-based, READ ONLY

for (int &x : v) x = x * 2;                           // range-based, MODIFIES v
```

Sort

```
sort(v.begin(), v.end());                  // ascending
sort(v.begin(), v.end(), greater<int>());  // descending
reverse(v.begin(), v.end());               // reverse order
```

2D Vector access (grid problems)

```
vector<vector<int>> grid = {{1,2},{3,4}};
grid[0][1]        // = 2  (row 0, col 1)
grid.size()       // number of ROWS
grid[0].size()    // number of COLUMNS
```

MAP / UNORDERED_MAP

Declare

```
map<int,int> m;              // sorted by key, slower
unordered_map<int,int> m;    // NOT sorted, faster - use this 90% of time
```

Insert / Update

```
m[5] = 10;             // set key 5 to value 10
m[5]++;                // increment value at key 5 (creates with 0 if not exists)
m.insert({5, 10});     // alternative insert syntax
```

Access

```
m[5]                  // get value at key 5 (creates key with 0 if missing - careful!)
m.at(5)               // get value, THROWS error if missing (safer for checking)
```

```
m.count(5)             // returns 1 if key exists, 0 if not  <- USE THIS to check existence
m.find(5)              // returns iterator, == m.end() if not found
```

Check if key exists (the pattern you'll use constantly)

```
if (m.count(5)) {
    cout << "key exists, value is " << m[5];
} else {
    cout << "key does not exist";
}

// Two Sum pattern - exact syntax:
unordered_map<int,int> seen;
for (int i = 0; i < nums.size(); i++) {
    int need = target - nums[i];
    if (seen.count(need)) return {seen[need], i};
    seen[nums[i]] = i;
}
```

Loop through map

```
for (auto& [key, val] : m) {
    cout << key << " -> " << val;
}

// older syntax (works everywhere, use if above gives error)
for (auto it = m.begin(); it != m.end(); it++) {
    cout << it->first << " -> " << it->second;
}
```

Remove

```
m.erase(5);    // remove key 5
```

SET / UNORDERED_SET (for 'have I seen this before' problems)

```
unordered_set<int> s;
s.insert(5);
s.count(5);       // 1 if exists, 0 if not
s.erase(5);
for (int x : s) cout << x;     // loop through, no guaranteed order
```

STRING (since palindrome/string problems keep showing up)

```
string s = "hello";
s.size()              // length
s[0]                  // 'h' - access like array
s.substr(1, 3)        // "ell" - starts at index 1, length 3
s + "world"           // concatenation
s.push_back('!');     // add char to end
reverse(s.begin(), s.end());   // reverse the string
to_string(123)        // int -> string
stoi("123")           // string -> int
```

PAIR (used constantly in BFS / graph / two-sum style answers)

```
pair<int,int> p = {1, 2};
p.first      // 1
p.second     // 2

vector<pair<int,int>> v;
v.push_back({1,2});
v.push_back(make_pair(3,4));
```

Drill — type these from memory, no copy-paste

1. Declare a vector of size 10 filled with -1
2. Loop through a vector and print each element with index
3. Declare unordered_map and insert 3 key-value pairs
4. Check if key 'apple' exists in that map without crashing if missing
5. Declare a 2D vector grid of size 4x5 filled with 0
6. Sort a vector in descending order
7. Loop through an unordered_map and print key->value for all entries
8. Write the Two Sum function from memory using unordered_map